System Architecture & Infrastructure Design

Hasnahena Web Application

Version:	1.0
Date:	March 22, 2026
Status:	Approved
Author:	Md Saif Ahmed

Scalable, secure, and compliant platform architecture

CONFIDENTIAL — For Internal Use Only

Contents

Document Control **2**

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Architecture Principles . . . 3

2 Architecture Overview **3**
2.1 High-Level Data Flow . . . 3

3 Technology Stack **4**
3.1 Frontend Layer . . . 4
3.2 Backend Microservices . . . 4
3.3 Database & Storage . . . 4

4 Security & Third-Party Integrations **4**
4.1 Security Stack . . . 4
4.2 Third-Party Integrations . . . 5

5 Infrastructure & DevOps **5**
5.1 Cloud Strategy . . . 5
5.2 Observability . . . 5
5.3 CI/CD Pipeline . . . 5

6 Scaling & Migration Path **5**
6.1 Phase 1: MVP (Free Tier) . . . 5
6.2 Phase 2: Growth . . . 6
6.3 Phase 3: Enterprise . . . 6

Document Control

Item	Details
Document Title	System Architecture & Infrastructure Design
Version	1.0
Date	March 22, 2026
Author	Md Saif Ahmed
Reviewed By	—
Approved By	—

Revision History

Ver.	Date	Author	Changes
1.0	—	Md Saif Ahmed	Initial draft
2.0	—	Md Saif Ahmed	Added microservices design
3.0	March 22, 2026	Md Saif Ahmed	Final release

1 Introduction

1.1 Purpose

This document outlines the architectural design and technology stack for the Hasnahena cooperative platform. The architecture is designed to ensure:

- **Scalability:** Handle growth from hundreds to thousands of members
- **Security:** Protect sensitive financial data and prevent unauthorized access
- **Compliance:** Meet regulatory and Shariah requirements
- **Reliability:** Ensure high availability for financial operations

1.2 Architecture Principles

1. **Separation of Concerns:** Clear boundaries between frontend, API gateway, and core services
2. **Defense in Depth:** Multiple security layers from WAF to application-level RBAC
3. **Immutable Audit:** All financial actions are logged and cannot be modified
4. **Progressive Enhancement:** Start with managed services, migrate to self-hosted as scale requires

2 Architecture Overview

The system employs a **dual-microservice architecture**:

- **Go (Golang) Edge Service:** Handles high-throughput traffic, real-time WebSockets, and API gateway functions
- **Java (Spring Boot) Core Ledger:** Handles strict double-entry bookkeeping, ACID transactions, and complex financial calculations
- **Next.js Frontend:** Server-side rendered React application for optimal performance and SEO

2.1 High-Level Data Flow

1. User accesses the application via `hasnahena.org`
2. Cloudflare WAF filters malicious traffic
3. Request reaches Next.js frontend on Vercel
4. API calls route through Go Edge Gateway
5. Financial operations are delegated to Java Core Ledger
6. Data persists to PostgreSQL with full audit logging

3 Technology Stack

3.1 Frontend Layer

Component	Technology
Framework	Next.js 14+ (App Router) with React 18
Language	TypeScript (strict mode)
UI System	Tailwind CSS + Shadcn UI
State Management	Zustand + React Query
Internationalization	next-intl (English/Bengali)
Hosting	Vercel (Edge caching, zero-config deployments)

3.2 Backend Microservices

Service	Technology & Responsibility
API Gateway (Edge)	Go with Chi/Gin router — traffic handling, rate limiting, WebSockets, JWT validation
Core Ledger	Java 17+ with Spring Boot 3 — double-entry bookkeeping, ACID transactions, loan calculations
Internal Communication	gRPC (synchronous), Apache Kafka (async event streaming)
Hosting	Fly.io (initial), AWS EKS (production scale)

3.3 Database & Storage

Component	Technology
Primary Database	PostgreSQL 15+ (Neon Serverless → AWS RDS)
Cache Layer	Valkey (open-source Redis alternative)
Document Storage	Cloudinary (MVP), AWS S3 (production)
ORM	Prisma (Go edge), Hibernate (Java core)

4 Security & Third-Party Integrations

4.1 Security Stack

Layer	Implementation
Identity & Auth	Clerk/Auth.js (Phase 1), Keycloak (enterprise rollout)
WAF / DDoS	Cloudflare Pro/Enterprise
Secrets Management	HashiCorp Vault (Community Edition)
TLS/SSL	Cloudflare managed certificates
RBAC	Custom implementation with JWT claims

4.2 Third-Party Integrations

Service	Provider
NID Verification	Porichoy API (Bangladesh NID Gateway)
SMS Gateway	SSL Wireless (local Bangladesh aggregator)
Email	Resend (transactional emails)
Domain	Namecheap with Cloudflare DNS

5 Infrastructure & DevOps

5.1 Cloud Strategy

Phase	Infrastructure
MVP / Launch	Vercel (frontend), Fly.io (backend), Neon (database)
Production Scale	Vercel (frontend), AWS EKS (backend), AWS RDS (database)
Region	ap-south-1 (Mumbai) or ap-southeast-1 (Singapore) for Bangladesh proximity

5.2 Observability

Function	Tool
Metrics & Dashboards	Prometheus + Grafana
Application Tracing	SigNoz (open-source)
Log Aggregation	Grafana Loki
Alerting	Grafana Alertmanager

5.3 CI/CD Pipeline

- **Source Control:** GitHub (separate repos for frontend and backend)
- **CI/CD:** GitHub Actions
- **Containerization:** Docker with multi-stage builds
- **Orchestration:** Kubernetes (AWS EKS in production)
- **Deployment:** GitOps with automatic deploys on `main` branch

6 Scaling & Migration Path

6.1 Phase 1: MVP (Free Tier)

- Vercel Hobby plan (frontend)
- Fly.io free tier (Go backend)
- Neon free tier (0.5 GB database)
- Cloudinary free tier (images)
- Estimated monthly cost: **$0-10**

6.2 Phase 2: Growth

- Vercel Pro ($20/month)
- Fly.io scaled instances ($15-30/month)
- Neon Launch plan ($19/month)
- Estimated monthly cost: **$50-100**

6.3 Phase 3: Enterprise

- AWS EKS with auto-scaling
- AWS RDS Multi-AZ PostgreSQL
- Cloudflare Enterprise
- Full observability stack
- Estimated monthly cost: **$300-500+**